



1주차 발표

DL 1팀 노현선 엄지민 이민경 정희진

목차

1장. 머신 러닝과 딥러닝

#1.1 인공지능, 머신러닝과 딥러닝

#1.2 머신러닝이란

#1.3 딥러닝이란

2장. 실습 환경 설정과 파이토치 기초

#2.1 파이토치 개요

#2.2 파이토치 기초 문법

#2.3 실습 환경 설정

#2.4 파이토치 코드 맛보기



1.1 인공지능, 머신러닝과 딥러닝



#1.1 인공지능, 머신러닝과 딥러닝

인공지능

인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술

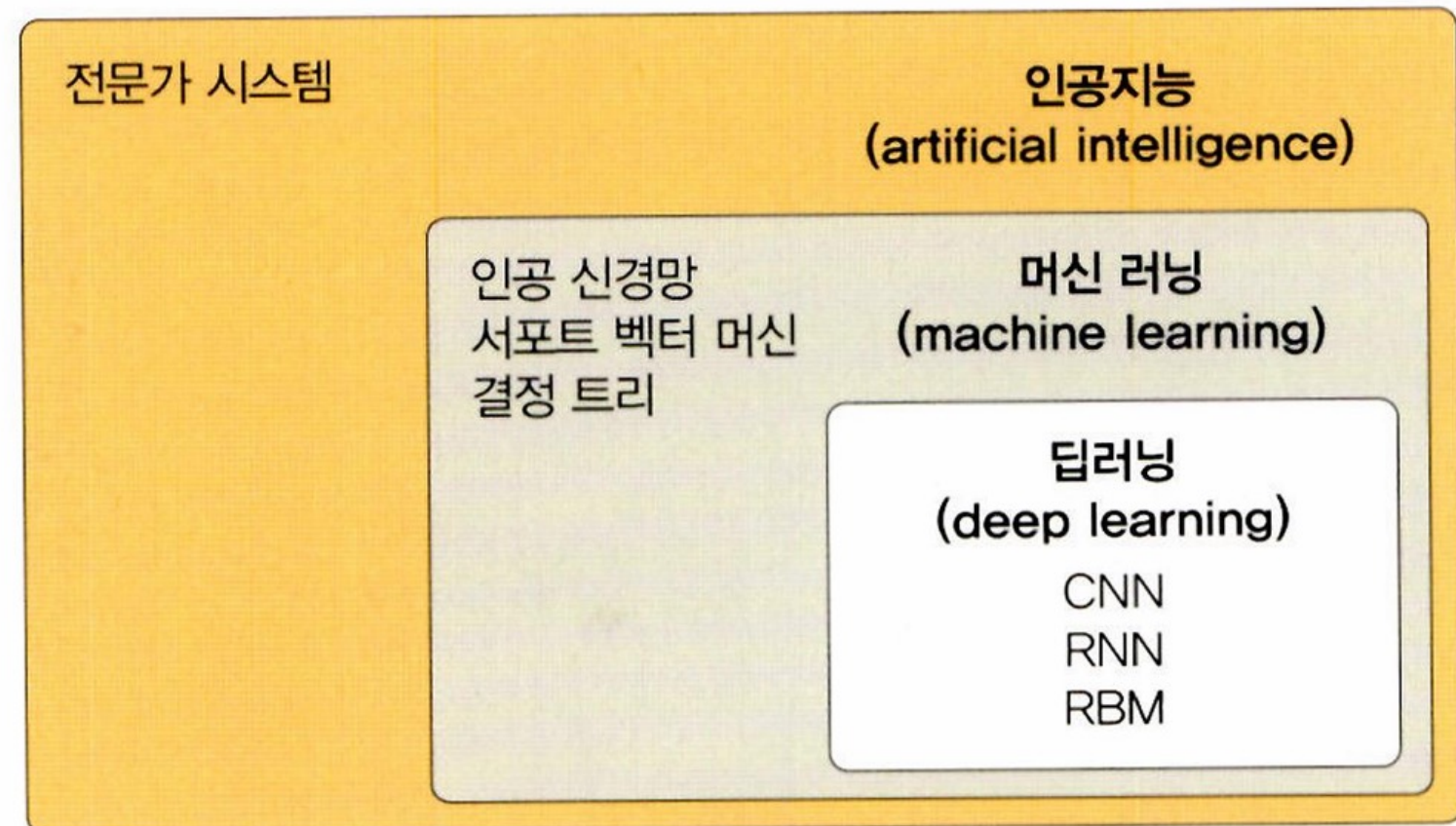
인공지능을 구현하는 방법: 머신러닝, 딥러닝

머신러닝

컴퓨터에서 스스로 대규모 데이터에서 지식, 패턴을 학습하고 예측하는 알고리즘 분야

딥러닝

신경망(CNN, RNN 등)을 활용하여 데이터 특징을 자동 추출, 학습하는 머신러닝 하위 분야



#1.1 인공지능, 머신러닝과 딥러닝

머신러닝 vs 딥러닝

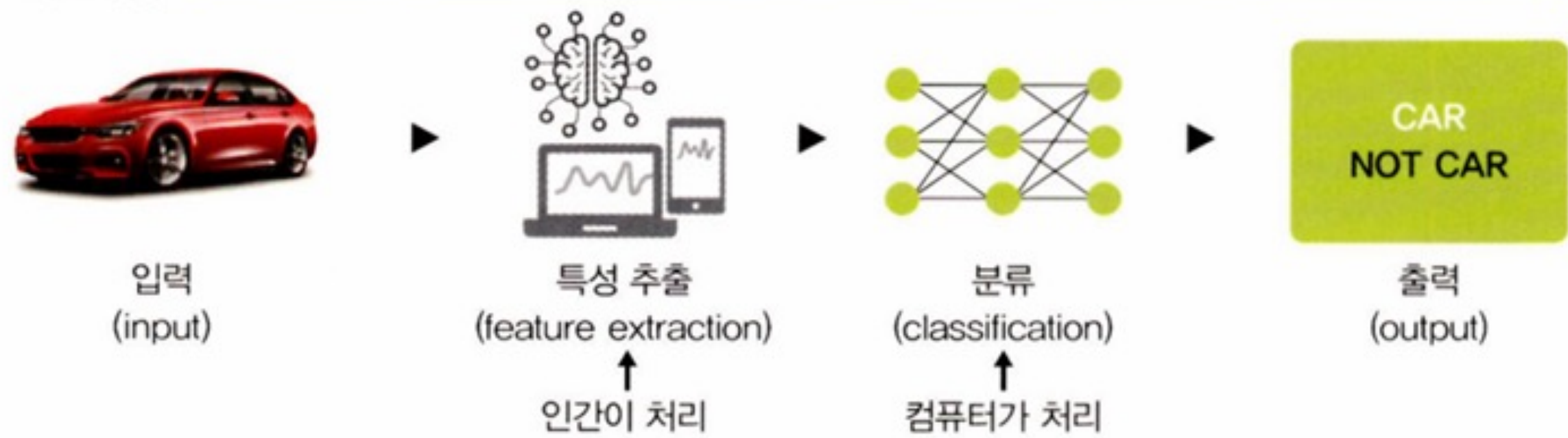
구분	머신러닝	딥러닝
동작 원리	알고리즘으로 예측 수행/ 인간이 특성 처리	신경망으로 데이터 특징, 관계 자동 해석
데이터 규모	수천 개 수준	수백만 개 이상
훈련 시간	단시간	장시간
결과 형태	점수, 분류 등 숫자 값	텍스트, 소리, 이미지 등 다양
재사용성	같은 유형 데이터만 재사용 가능	동일 유형 데이터 재사용 가능

머신 러닝은 인간이 특성 설계 → 딥러닝은 컴퓨터가 특성을 스스로 추출

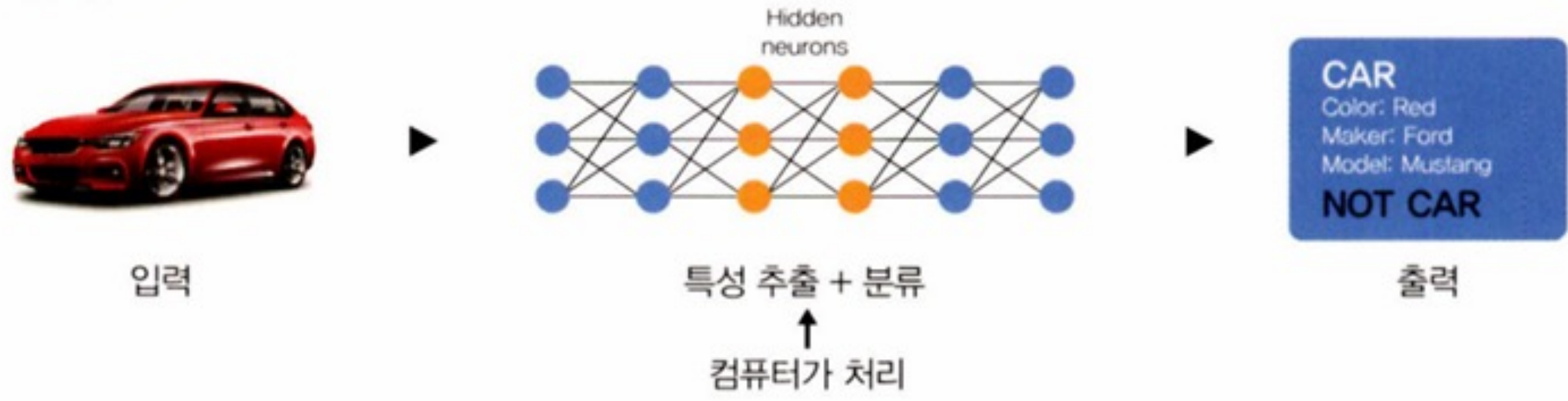
#1.1 인공지능, 머신러닝과 딥러닝

머신러닝 vs 딥러닝

머신 러닝



딥러닝



1.2 머신러닝이란



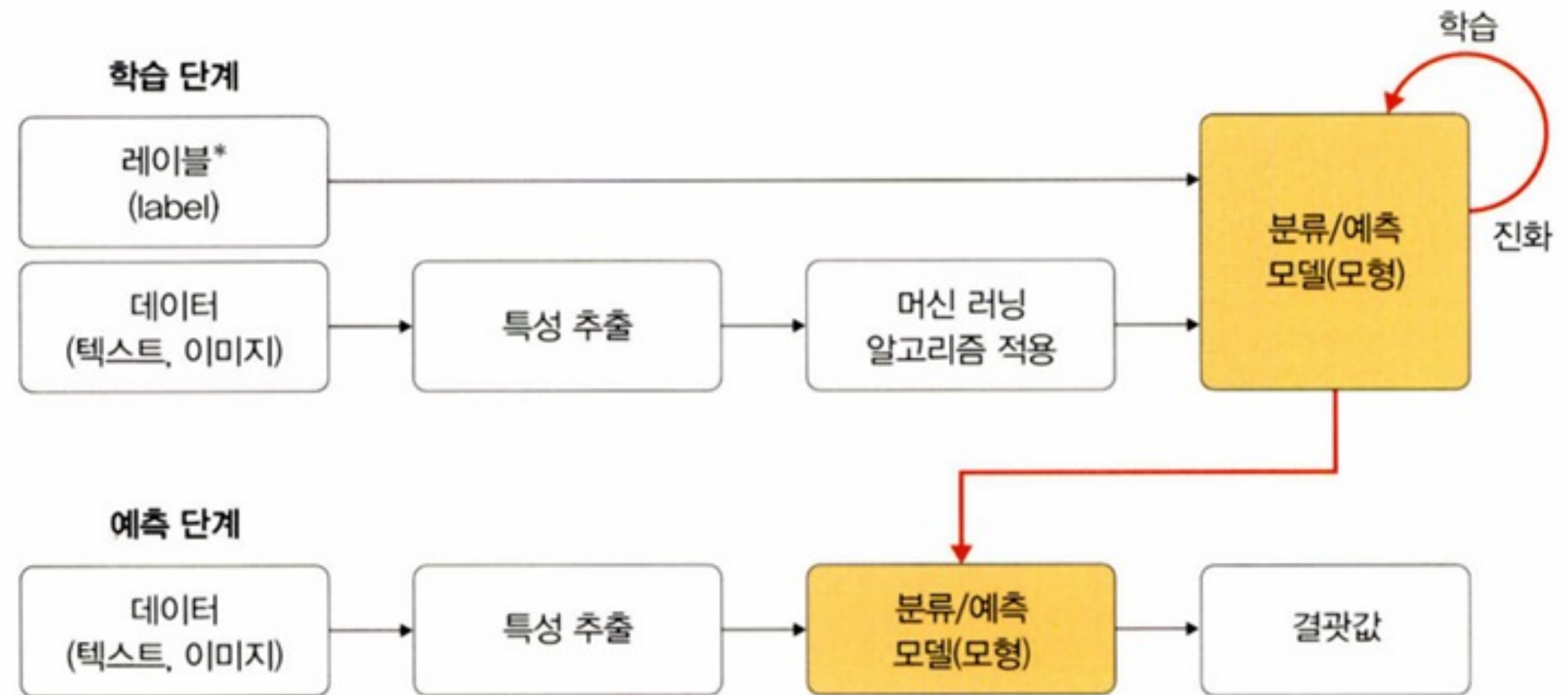
#1.2 머신러닝이란

머신러닝은 인공지능의 한 분야로, 컴퓨터 스스로 대량의 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행하는 것

머신 러닝 학습 과정 (2단계)

1. 학습 단계(Training)

- Label + data
- Feature Extraction
- ML 알고리즘 적용
- 분류/예측 모델 생성



* 레이블은 지도 학습에서 정답을 의미

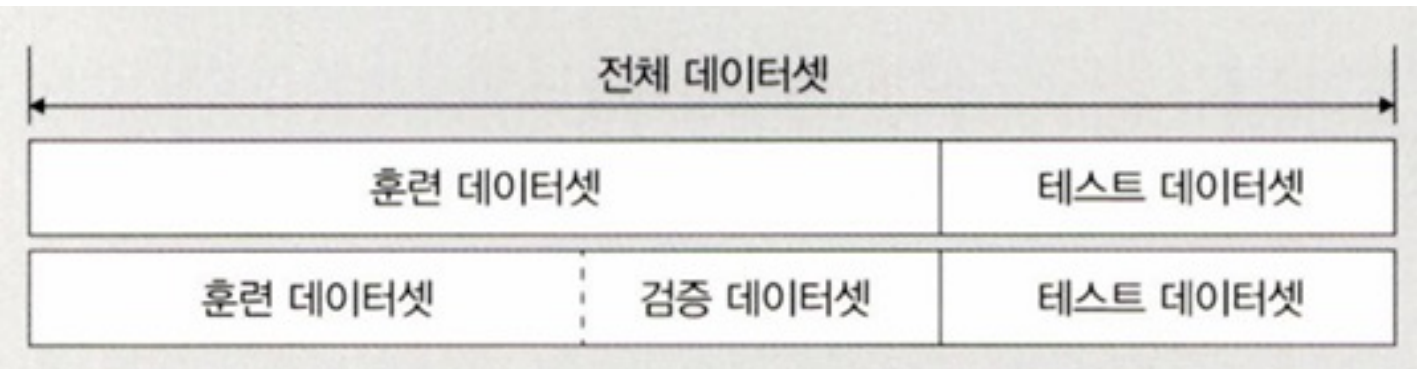
2. 예측 단계(Prediction)

- 새로운 데이터 입력
- 학습된 모델에 적용
- 결괏값(분류/예측) 출력

#1.2 머신러닝이란

데이터셋 구분 & 모델 학습 절차

데이터셋 분할

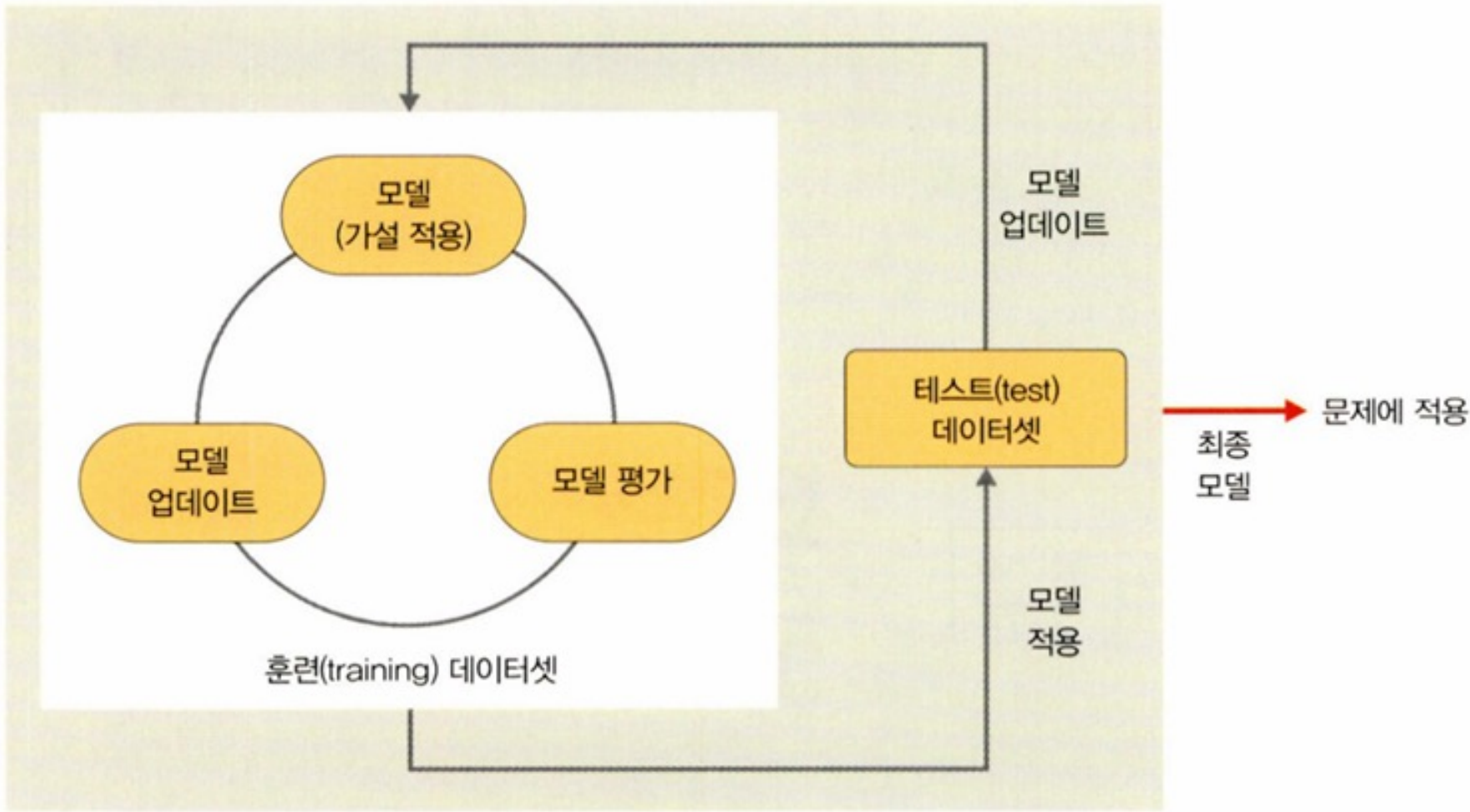


수집된 데이터는 보통 훈련(80%) / 테스트(20%) 로 나누며,
필요시 훈련 데이터를 다시 훈련 + 검증 으로 분리

검증 데이터셋

- 학습 중 모델 성능 점검
- 과적합(overfitting) 방지

모델 학습 절차



#1.2 머신러닝이란

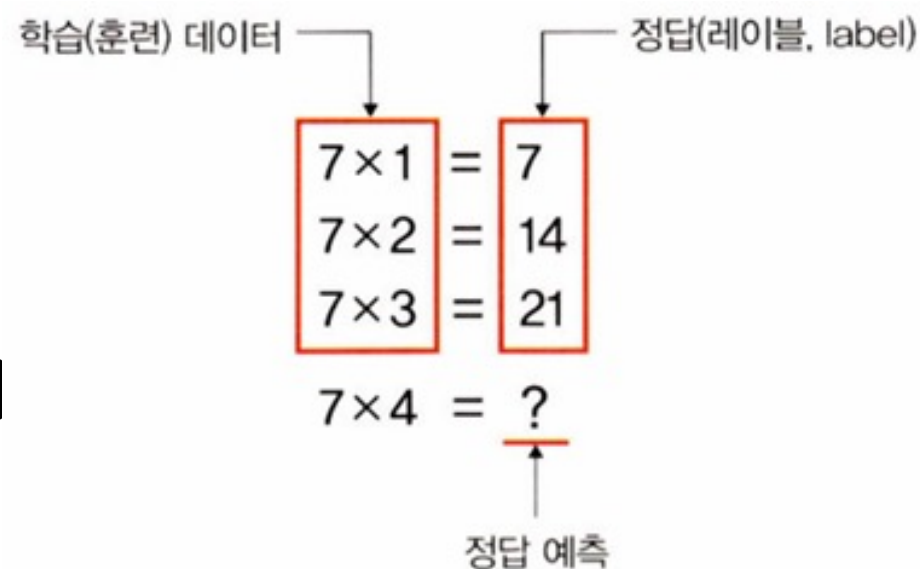
머신 러닝 학습 알고리즘

지도 학습 (Supervised)

정답(label)을 알려주고 학습시키는 방법

분류: KNN, SVM, 결정 트리, 로지스틱 회귀

회귀: 선형 회귀



강화 학습 (Reinforcement)

행동에 대한 보상을 받으며 스스로 학습

에이전트가 환경과 상호작용

보상 극대화를 목표로 행동 선택

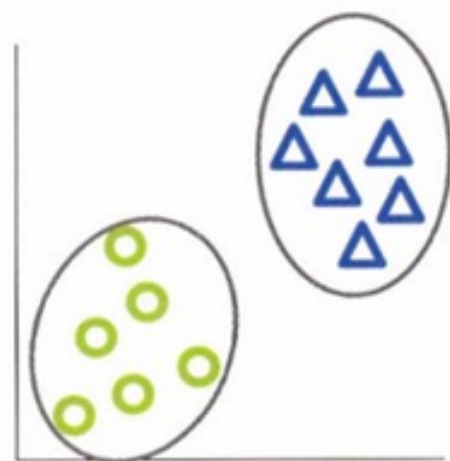
대표 알고리즘: 마르코프 결정 과정(MDP)

비지도 학습 (Unsupervised)

정답 없이 유사 데이터끼리 스스로 분류

군집화: K-평균, DBSCAN

차원 축소: 주성분 분석(PCA)



1.3 답러닝이란



#1.3 딥러닝이란

머신 러닝과 딥러닝의 차이

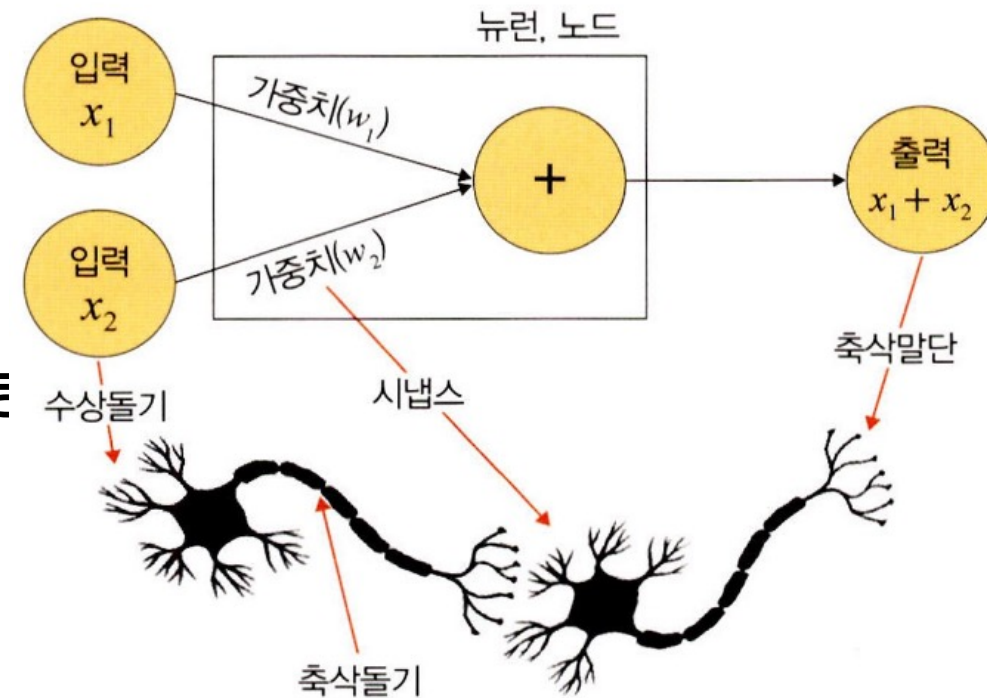
딥러닝은 인간의 뇌를 기초로 설계하였음

-> 뉴런과 시냅스에서 착안 하여

-> 수많은 뉴런을 '병렬 연산'

-> 기존 컴퓨터가 수행하지 못한 음성, 인식 처리 가능

▼ 그림 1-10 인간의 신경망 원리를 모방한 심층 신경망



- **수상돌기:** 주변이나 다른 뉴런에서 자극을 받아들이고, 이 자극들을 전기적 신호 형태로 세포체와 축삭돌기로 보내는 역할을 합니다.
- **시냅스:** 신경 세포들이 이루는 연결 부위로, 한 뉴런의 축삭돌기와 다음 뉴런의 수상돌기가 만나는 부분입니다.
- **축삭돌기:** 다른 뉴런(수상돌기)에 신호를 전달하는 기능을 하는 뉴런의 한 부분입니다. 뉴런에서 뻗어 있는 돌기 중 가장 길며, 한 개만 있습니다.
- **축삭말단:** 전달된 전기 신호를 받아 신경 전달 물질을 시냅스 틈새로 방출합니다.

#1.3.1 딥러닝 학습과정

1. 데이터 준비

파이토치 에서 제공하는 데이터 셋 활용
캐들과 같은 공개된 데이터 셋 활용
-> 전처리가 이미 되어 있어 편함.

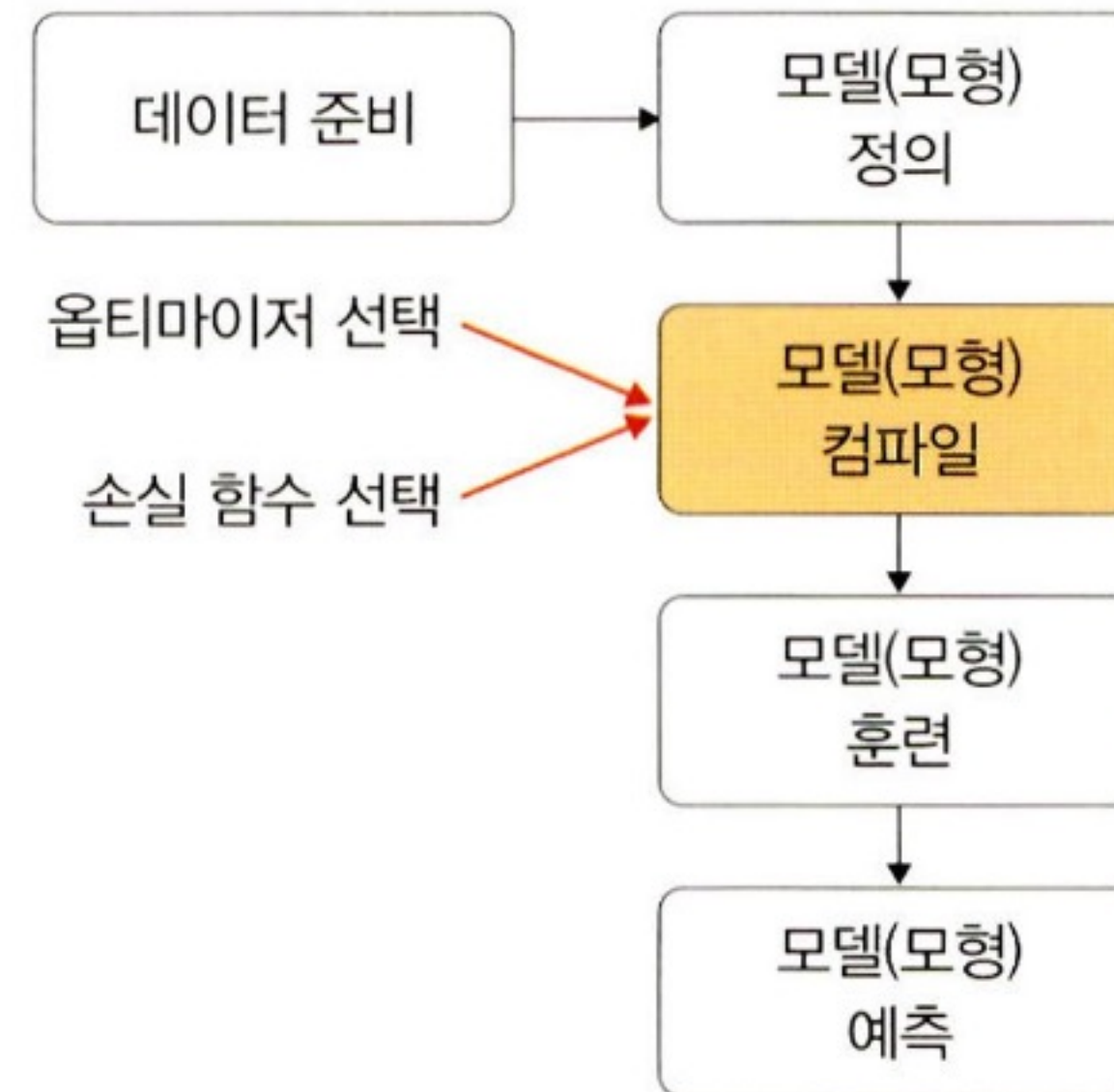
2. 모델(모형) 정의

신경망 형성
은닉층 개수가 많을수록 성능이 좋아짐
그러나 과적합 가능성 높아짐

* 과적합 방지 방법

1. 데이터 증강
2. 드롭아웃 (일부 뉴런 끄고 학습하기)
3. 가중치 규제
4. 조기 종료
5. 은닉층 줄이기
6. 배치 정규화

▼ 그림 1-11 딥러닝 모델의 학습 과정



#1.3.1 딥러닝 학습과정

3. 모델(모형) 컴파일

활성화 함수: 입력 신호가 일정 기준 이상이면 출력 신호로 변환

ex) 시그모이드, 탄젠트,렐루

손실함수: 모델 출력값과 사용자가 원하는 출력값 차이를 구하는 함수

ex) 평균 제곱 오차, 엔트로피 오차

옵티마이저: 손실 함수를 기반으로 네트워크 업데이트 방법 결정.

ex) 아담, 알엠에스,프롭 등

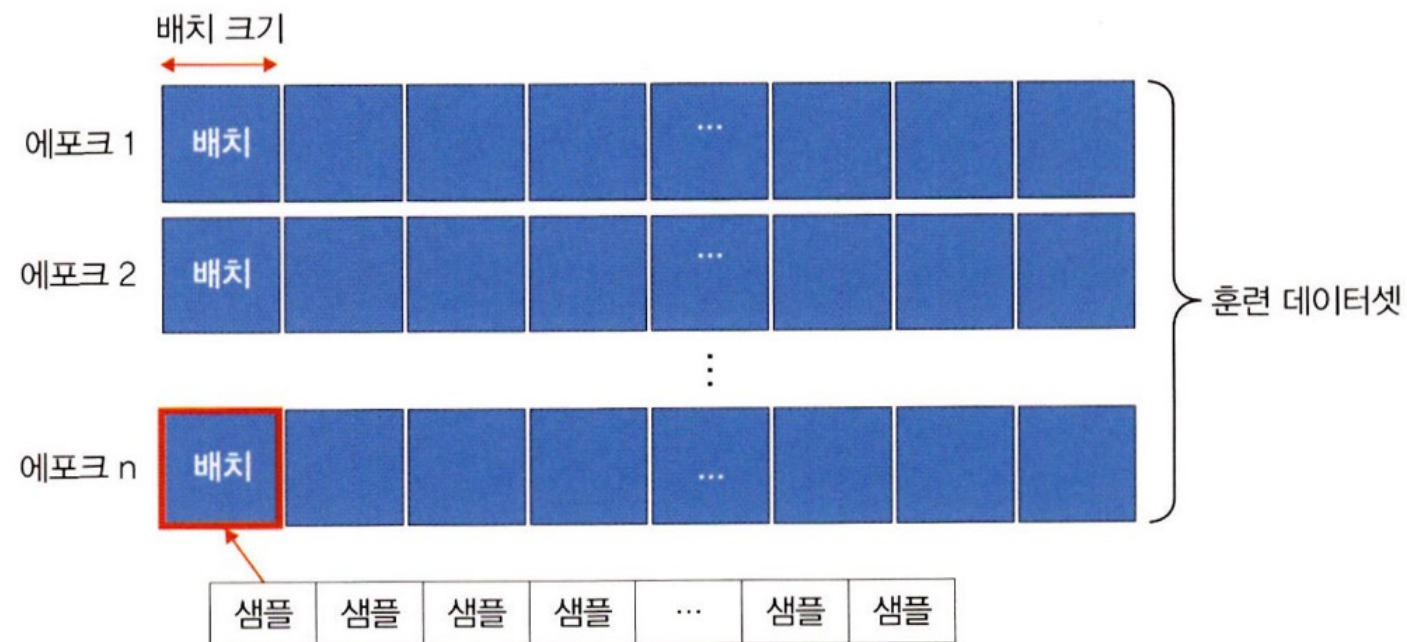
4. 모델(모형) 훈련

한 번에 처리할 데이터양 -> 배치
훈련 횟수 -> 에포크 을 지정해야 함.

처리량이 많을 경우,
학습 속도 느려짐
메모리 부족 문제 야기

따라서 눈으로 확인하면서 최적 값을 찾아야 함.

▼ 그림 1-12 모델 훈련에 필요한 하이퍼파라미터



#1.3.1 딥러닝 학습과정

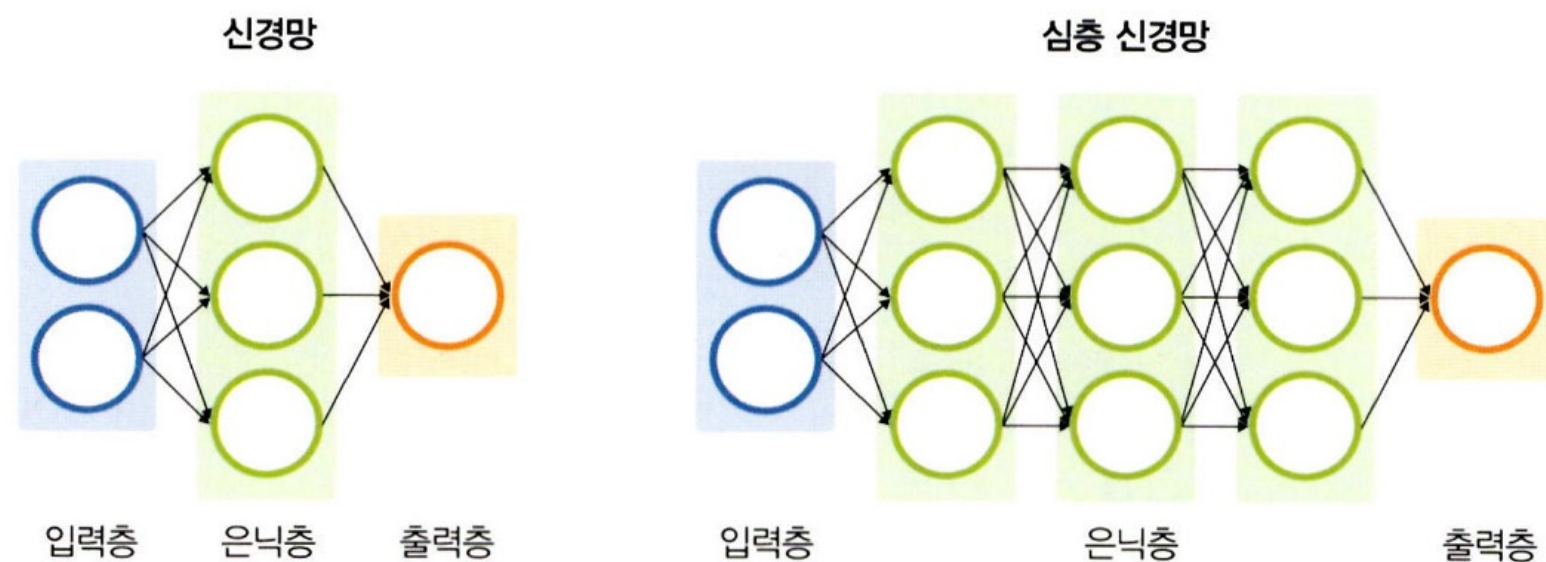
5. 모델(모형) 예측

신경망과 역전파

심층 신경망:

데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있음.

▼ 그림 1-13 신경망과 심층 신경망



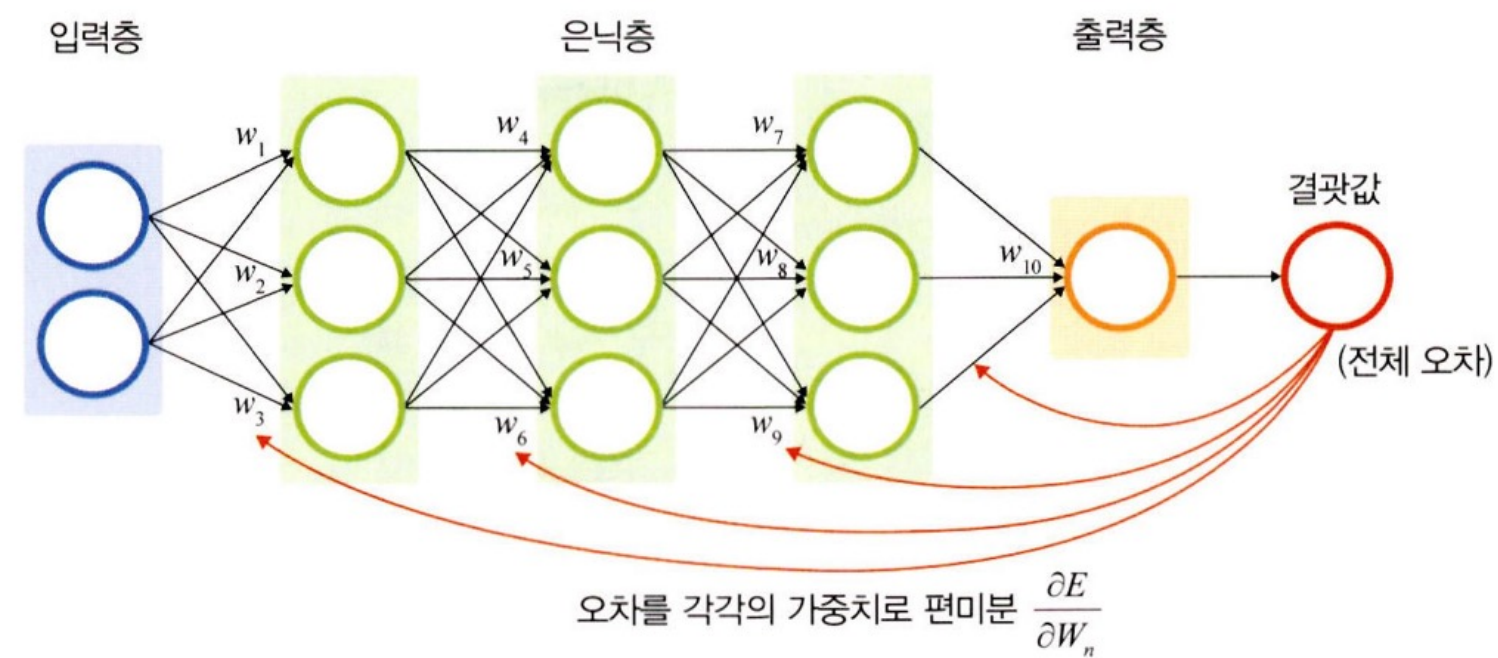
(오차)역전파:

신경망의 가중치 또는 편향에 대한 변경이 모델 예측의 정확도에 어떤 영향을 미치는지 계산하는 효율적인 방법

특히 계산 과정에서 사용되는 미분이 성능에 영향을 미침.

-> 파이토치를 사용하면 역전파 알고리즘을 자동으로

▼ 그림 1-14 역전파 계산



#1.3.2 딥러닝 학습 알고리즘

지도 학습, 비지도 학습, 전이 학습으로 나뉨

1. 지도 학습 알고리즘

합성곱 신경망(CNN)

이미지 분류, 객체를 식별하는 컴퓨터 비전 기술에서 가장 많이 사용됨.

종류: 이미지 분류, 이미지 인식(질병 식별, 산업 검사 등), 이미지 분할.(객체 간 영역을 픽셀 단위로 구분)

순환 신경망(RNN)

시계열 데이터 분류할 활용.

문제점: 역전파 과정에서 기울기 소멸 문제 발생

-> LSTM: 망각게이트-입력게이트-출력 게이트를 도입하여 소멸 문제 해결.

2. 비지도 학습 알고리즘

워드 임베딩

자연어 처리에 있어 단어를 벡터로 표현.
워드투벡터와 글로브를 통해 단어 의미를 벡터화

군집

한 클러스터 안의 데이터는 비슷, 다른 데이터와는 구분.

머신러닝과 크게 다르지 않으나 성능이 더 좋음.

#1.3.2 딥러닝 학습 알고리즘

지도 학습, 비지도 학습, 전이 학습으로 나뉨

3. 전이학습

사전 학습이 완료된 모델을 활용. 미세 조정 기법을 이용하여 학습시키는 방법

-> 자원을 줄이고 효율적인 학습 가능

Ex) VGG, 인셉션(Inception), MobileNet 등

-> 미세 조정 기법을 활용하는 방법을 배우는 것이 중요

.

▼ 표 1-3 지도 학습, 비지도 학습, 강화 학습

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습(unsupervised learning)	군집 (clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	-	마르코프 결정 과정(MDP)

2.1 파이토치 개요



#2.1.1 파이토치 개요·특징 및 장점

📌 파이토치 개요

- 2017년 공개된 딥러닝 프레임워크
- 파이썬 기반의 과학 연산 패키지
 - 파이썬의 NumPy를 대체하며 GPU를 이용한 연산이 필요할 때
 - 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요할 때

📌 파이토치 특징

“GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크”

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$$

1. GPU: 연산 속도를 높이는 역할
2. 텐서: 파이토치의 데이터 형태로, 단일 데이터 형식으로 된 자료들의 다차원 행렬 (3차원 이상)
3. 동적 신경망: 훈련 반복 시마다 네트워크 변경이 가능한 신경망

#2.1.1 파이토치 개요·특징 및 장점

📌 텐서 (Tensor)

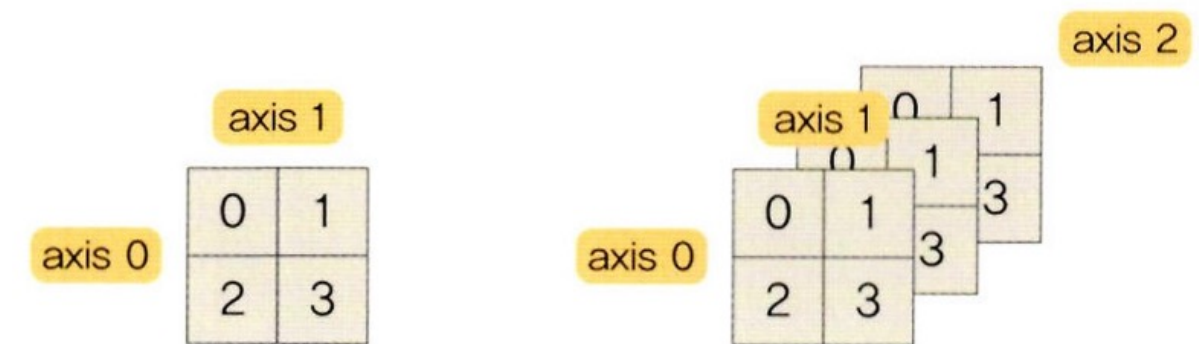
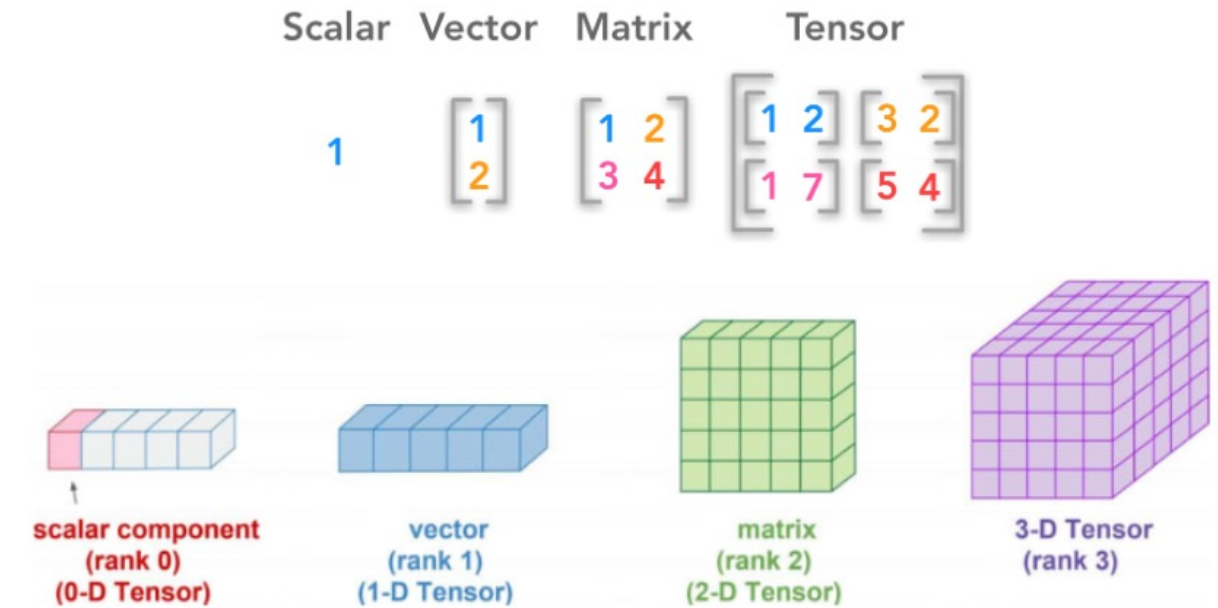
- 행렬을 여러 개 쌓아 올린 다차원 표현 (3차원 이상)
- 벡터, 행렬, 텐서의 비교
 - 벡터: 1차원 벡터 형태의 숫자들로 이루어진 리스트
 - 행렬: 행과 열로 표현되는 2차원 배열 형태

*`torch.tensor()` 사용하여 텐서 표현

```
import torch
```

```
torch.tensor([[1., -1.], [1., -1.]])
```

```
tensor([[ 1., -1.],  
        [ 1., -1.]])
```



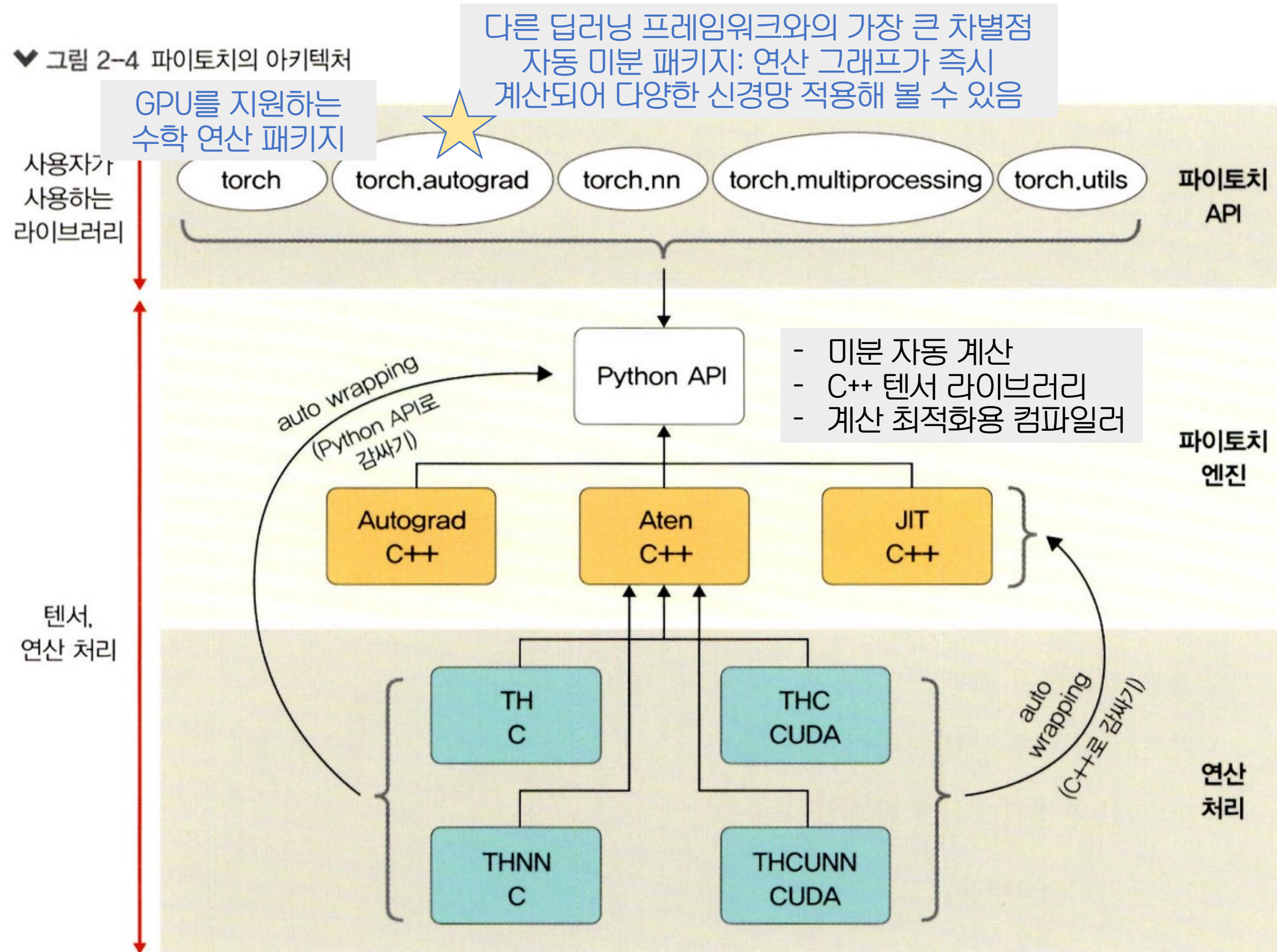
*axis 0: 벡터, axis 1: 행렬, axis 2: 텐서

#2.1.1 파이토치 개요·특징 및 장점

📌 파이토치의 장점

1. 효율적 계산
 - 파이썬 환경과의 통합 용이, 직관적 디버깅
2. 고성능: 낮은 CPU 활용
 - 모델 훈련에 사용되는 CPU 사용률 낮음
 - 학습 및 추론 속도 빠름
3. 직관적 인터페이스

#2.1.2 파이토치의 아키텍처



텐서에 대한 처리와 신경망 구축을
훈련할 수 있도록 사용자 친화형
API 제공 (실제 계산은 하지 않음)

C / Cuda 패키지
상위 API에서 할당된 거의 모든
계산 수행: 효율적 데이터 구조 및
다차원 텐서에 대한 연산 처리

2.2 파이토치 기초 문법

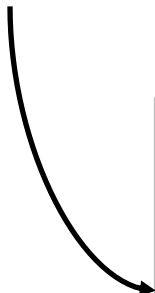


#2.2.1 텐서 다루기

📌 텐서 생성 및 변환

텐서를 다루는 기초 문법: 딥러닝 모델의 입출력을 제어하기 때문에 중요도 높음.

```
import torch
print(torch.tensor([[1,2],[3,4]])) ----- 2차원 형태의 텐서 생성 (CPU 메모리에 생성)
print(torch.tensor([[1,2],[3,4]], device="cuda:0")) ----- GPU에 텐서 생성 병렬 연산 가속:  $y=wx+b$  계산을 동시에 처리하여 CPU보다 훨씬 빠름
print(torch.tensor([[1,2],[3,4]], dtype=torch.float64)) ----- dtype을 이용하여 텐서 생성
```



```
tensor([[1, 2],
        [3, 4]])
tensor([[1., 2.],
        [3., 4.]], dtype=torch.float64)
```

* 텐서 > ndarray 변환하기

```
temp = torch.tensor([[1,2],[3,4]])
print(temp.numpy()) ----- 텐서를 ndarray로 변환

temp = torch.tensor([[1,2],[3,4]], device="cuda:0")
print(temp.to("cpu").numpy()) ----- GPU상의 텐서를 CPU의 텐서로 변환한 후 ndarray로 변환
```

#2.2.1 텐서 다루기

📌 텐서의 인덱스 조작

- ndarray 조작과 유사하게 동작: 인덱스 직접 지정 및 슬라이스 등 사용 가능
- 텐서의 자료형
 - torch.FloatTensor: 32bit 부동 소수점
 - torch.DoubleTensor: 64bit 부동 소수점
 - torch.LongTensor: 64bit 부호가 있는 정수

```
temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7]) ----- 파이토치로 1차원 벡터 생성
print(temp[0], temp[1], temp[-1]) ----- 인덱스로 접근
print('-----')
print(temp[2:5], temp[4:-1]) ----- 슬라이스로 접근
```

-1번 인덱스: 맨 뒤에서부터 시작하는 인덱스

```
tensor(1.) tensor(2.) tensor(7.)
-----
tensor([3., 4., 5.]) tensor([5., 6.])
```

#2.2.1 텐서 다루기

📌 텐서 연산

- ndarray 조작과 유사하게 다양한 수학 연산 가능: GPU 사용 시 속도 향상
- 텐서 간 타입이 일치해야 연산 수행

```
v = torch.tensor([1, 2, 3]) ----- 길이가 3인 벡터 생성
w = torch.tensor([3, 4, 6])
print(w - v) ----- 길이가 같은 벡터 간 뺄셈 연산
```

output: tensor([2, 2, 3])

📌 텐서 차원 조작

- (1) view: 가장 대표적인 방법으로 numpy의 reshape과 유사하게 텐서의 모양 변경
- (2) transpose, t: 차원 교환
- (3) stack, cat: 텐서 결합

#2.2.1 텐서 다루기

📌 텐서 차원 조작

```
temp = torch.tensor([
    [1, 2], [3, 4]]) ----- 2x2 행렬 생성

print(temp.shape)
print('-----')
print(temp.view(4, 1)) ----- 2x2 행렬을 4x1로 변형
print('-----')
print(temp.view(-1)) ----- 2x2 행렬을 1차원 벡터로 변형
print('-----')
print(temp.view(1, -1)) ----- -1은 (1, ?)와 같은 의미로 다른 차원으로부터 해당 값을 유추하겠다는
                               ----- 것입니다. temp의 원소 개수(2x2=4)를 유지한 채 (1, ?)의 형태를 만
                               ----- 족해야 하므로 (1, 4)가 됩니다.
print('-----')
print(temp.view(-1, 1)) ----- 앞에서와 마찬가지로 (?, 1)의 의미로 temp의 원소 개수(2x2=4)를
                               ----- 유지한 채 (?, 1)의 형태를 만족해야 하므로 (4, 1)이 됩니다.
```

output

```
torch.Size([2, 2])
-----
tensor([[1],
        [2],
        [3],
        [4]])
-----
tensor([1, 2, 3, 4])
-----
tensor([[1, 2, 3, 4]])
-----
tensor([[1],
        [2],
        [3],
        [4]])
```

#2.2.2 데이터 준비

📌 파이토치에서 데이터를 불러오는 다양한 방법

대량의 데이터를 효율적으로 처리하기 위해 표준화된 데이터 로딩 방식을 제공

1. **Custom Dataset**: Dataset 클래스를 상속받아 `__init__`, `__len__`, `__getitem__`을 구현하여 데이터를 조금씩 나누어 호출
2. **DataLoader**: 데이터셋을 배치 단위로 쪼개고 데이터를 섞는 역할
3. **Torchvision**: 파이토치 제공, MNIST, ImageNet 등 유명한 데이터셋을 쉽게 내려받을 수 있는 패키지

*데이터셋 호출 방법 및 구현은 교재 예제 참고

#2.2.3 모델 정의

📌 모델 정의 및 구조

모델: 계층(Layer)과 모듈이 모여 구성됨.

1. 계층: 모듈 또는 모듈을 구성하는 한 개의 계층. 합성곱층, 선형계층 등이 있음.
2. 모듈: 한 개 이상의 계층이 모여 구성된 것. 모듈이 모여 새로운 모듈을 생성할 수 있음.
3. 모델: 최종적으로 원하는 네트워크.

📌 파이토치에서 모델을 정의하는 세 가지 방법

1. nn.module 상속 (가장 권장되는 정석 방법)
2. nn.Sequential 사용 (가독성 중심)
3. 함수로 정의 (재사용성 중심)

#2.2.3 모델 정의

1. nn.module 상속 (가장 권장되는 정석 방법)

```
class MLP(Module):
```

```
    def __init__(self, inputs):  
        super(MLP, self).__init__()  
        self.layer = Linear(inputs, 1) ----- 계층 정의  
        self.activation = Sigmoid() ----- 활성화 함수 정의
```

```
    def forward(self, X):  
        X = self.layer(X)  
        X = self.activation(X)  
        return X
```

- `__init__()`: 구조 정의. 모델에서 사용할 계층과 활성화 함수 등을 미리 선언.
- `forward()`: 연산 수행. 입력 데이터가 각 계층을 어떤 순서로 통과해 결과가 나오는지 연산 과정 정의.

#2.2.3 모델 정의

2. Sequential 신경망 정의: nn.Sequential

- nn.Sequential: nn.module의 `__init__()`와 `forward()` 함수가 하는 역할 + 가독성 향상
- 코드가 간결하고 가독성이 높음
- 내부에 포함된 각 모듈을 순차적으로 실행: init에서 네트워크를 리스트처럼 묶어 정의하면 forward에서 각 모듈을 하나씩 호출할 필요 없이 순차적으로 자동 수행
- 계층이 복잡할수록 효과 뛰어남

```
import torch.nn as nn
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=64, out_channels=30, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer3 = nn.Sequential(
            nn.Linear(in_features=30*5*5, out_features=10, bias=True),
            nn.ReLU(inplace=True))

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = x.view(x.shape[0], -1)
```

```
self.layer2 = nn.Sequential(
    nn.Conv2d(in_channels=64, out_channels=30, kernel_size=5),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2))

self.layer3 = nn.Sequential(
    nn.Linear(in_features=30*5*5, out_features=10, bias=True),
    nn.ReLU(inplace=True))
```

합성곱(Conv2d), 활성화 함수(ReLU) 등과 같이 세트로 움직이는 하나의 연산들을 하나의 덩어리로 묶음

> 이후 모델 구조 수정 및 특정 블록 재사용 용이해짐

```
def forward(self, x):
    x = self.layer1(x)
    x = self.layer2(x)
    x = x.view(x.shape[0], -1)
```

모든 계층을 따로 정의했다면 10줄이 넘었을 forward 연산이 `self.layer1(x)`, `self.layer2(x)`처럼 블록 단위로 호출: 코드 간결해짐

#2.2.3 모델 정의

3. 함수로 정의

- Sequential 이용하는 것과 동일 but 함수 선언 시 변수에 저장해 둔 계층들 재사용 가능
- 모델이 복잡해진다는 단점도 있어서 복잡한 모델의 경우 nn.Module() 상속이 더 편리

```
def MLP(in_features=1, hidden_features=20, out_features=1):  
    hidden = nn.Linear(in_features=in_features, out_features=hidden_features,  
                        bias=True)  
    activation = nn.ReLU()  
    output = nn.Linear(in_features=hidden_features, out_features=out_features,  
                       bias=True)  
    net = nn.Sequential(hidden, activation, output)  
    return net
```

#2.2.4-5 학습 설정 및 프로세스

📌 모델 파라미터 정의

1. 손실 함수: 학습 중 출력과 실제 값 사이 오차 측정
2. 옵티마이저: 데이터와 손실 함수를 바탕으로 모델 업데이트 방식 결정
3. 학습률 스케줄러: 미리 지정한 횟수의 에포크를 지날 때마다 학습률 감소시킴으로써 최적점을 찾도록 함
4. 지표: 훈련과 테스트 단계 모니터링

```
from torch.optim import optimizer
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer=optimizer,
                                              lr_lambda=lambda epoch: 0.95 ** epoch)
```

📌 학습 5단계 절차

$y=wx+b$ 라는 함수의 적절한 w , b 값 갖도록 계속 수정 (전역 최소점에 이를 때까지)

1. optimizer.zero_grad(): 이전 단계의 기울기 초기화
2. output = model(input): 순전파 연산으로 출력값 계산
3. loss_fn(output, target): 실제값과의 오차 계산
4. loss.backward(): 역전파를 통해 기울기(Gradient) 계산
5. optimizer.step(): 계산된 기울기를 바탕으로 파라미터 업데이트

#2.2.6-7 모델 평가 및 훈련 과정 모니터링

📌 모델 평가

함수 혹은 모듈 이용하여 모델 평가 가능 (pip install torchmetrics 실행)

- torchmetrics.fuctional.accuracy(출력값, 정답) 함수
- metric = torchmetrics.Accuracy() 로 모듈을 객체 형태로 생성

📌 훈련 과정 모니터링 도구: Tensorflow

- 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 시각화
- 성능 추적 및 평가
- [텐서보드 설정 - 기록 - 모델 구조 살펴보기] 의 과정

2.3 실습 환경 설정



#2.3. 1 아나콘다 설치



Products ▾ Solutions ▾ Resources ▾ Company ▾

Sign In

Get Demo >

Windows

Mac

Linux

Get Started with Anaconda Distribution

Install Python, Jupyter, and thousands of data science packages in one step. Trusted by over 50 million users who need tools that work—without the setup headaches.

What's included in Anaconda Distribution?

- ✓ Thousands of vetted Python packages with dependency management
- ✓ Conda package and environment manager
- ✓ Jupyter Notebook and JupyterLab
- ✓ Desktop app (Navigator) or command line interface
- ✓ Works on Windows, macOS, and Linux

Download Now

Get access in 30 seconds. Completely free.*

Get Started >

Returning Users >

*Subject to our [Terms of Service](#). Use of Anaconda's offerings at an organization of more than 200 employees/contractors requires a paid business license unless your organization is eligible for discounted or free use. [See Pricing](#).

[Skip Registration >](#)

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

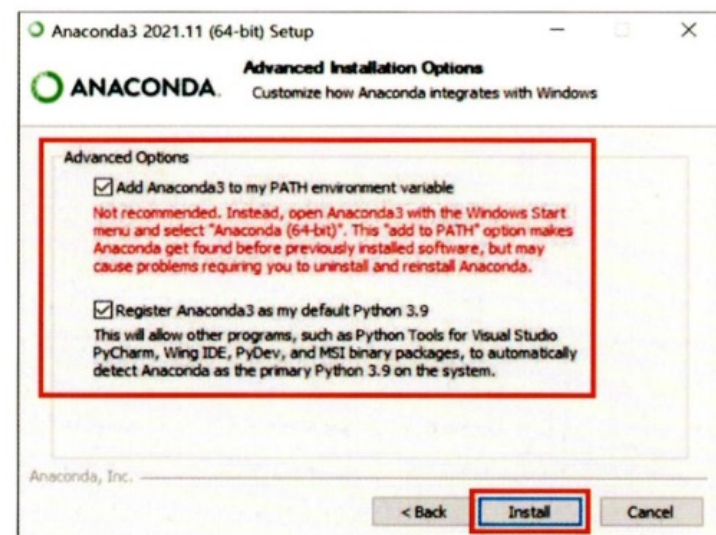
[64-Bit \(Apple silicon\) Graphical Installer](#)
[64-Bit \(Apple silicon\) Command Line Installer](#)

Miniconda

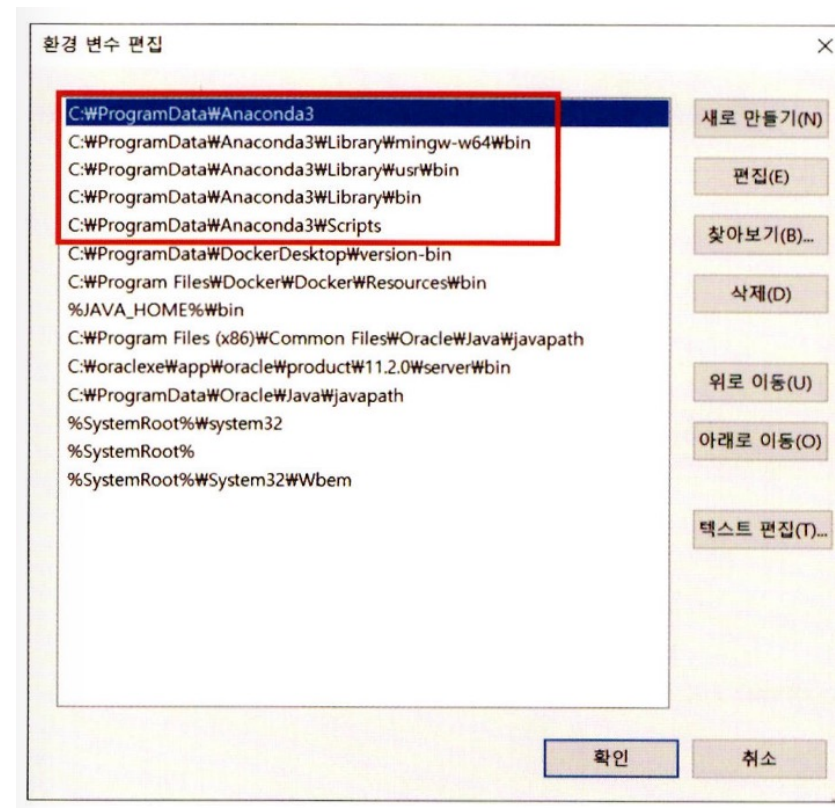
Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

[64-Bit \(Apple silicon\) Graphical Installer](#)
[64-Bit \(Apple silicon\) Command Line Installer](#)

적절한 버전 설치 진행



변수 등록을 위해 둘 다 체크해야함
(버전 차이인지 맥/윈도우 차이인지는
모르겠지만 없이 그냥 installer따라
진행



환경변수 확인:
전역 실행을 위해 환경
변수가 등록 돼있어야
함

맥북은 다음과 같은
명령어로 환경변수 확인
~ % printenv

#2.3. 2 가상 환경 생성 및 파이토치 설

치

파이토치: PC 직접 설치, 가상환경에 설치 둘 다 가능

직접 설치할 경우: 파이썬과 파이토치를 포함한 라이브러리 버전 관리 어려울 수 있으므로 가상 환경에 설치 진행

1. 가상 환경 생성

윈도우는 Anaconda Prompt, 맥북은 터미널에서 `conda create -n 환경이름 python = 3.9.0` (설치된 파이썬 버전에 따라 다름) 명령을 이용하여 가상 환경 생성. `conda create -n torch_book python=3.9`

`conda env list` 로 생성된 가상 환경 확인:

```
# conda environments:
#
# * -> active
# + -> frozen
base                *    /opt/anaconda3
torch_book           *    /opt/anaconda3/envs/torch_book
```

가상 환경 활성화: `conda activate 가상환경 이름`

가상환경 삭제: `conda env remove -n 가상환경이름`

생성된 가상 환경에 커널 설치: `conda install ipykernel`

가상 환경에 커널 연결: `ipython kernel install --name [사용자가 정한 커널 이름] --user`

주피터 노트북 접속: `jupyter notebook`

#2.3. 2 가상 환경 생성 및 파이토치 설

```
[(base) ~-MacBookPro ~ % ipython kernel install --name test_kernel --user
Installed kernelspec test_kernel in /Users/minlee/Library/Jupyter/kernels/test_kernel
[(base) ~-MacBookPro ~ % jupyter notebook
```

2. 파이토치 설치

CPU에서 설치하는 경우:

```
conda install pytorch==1.9.0 torchvision==0.10.0 torchaudio==0.9.0 -c pytorch
```

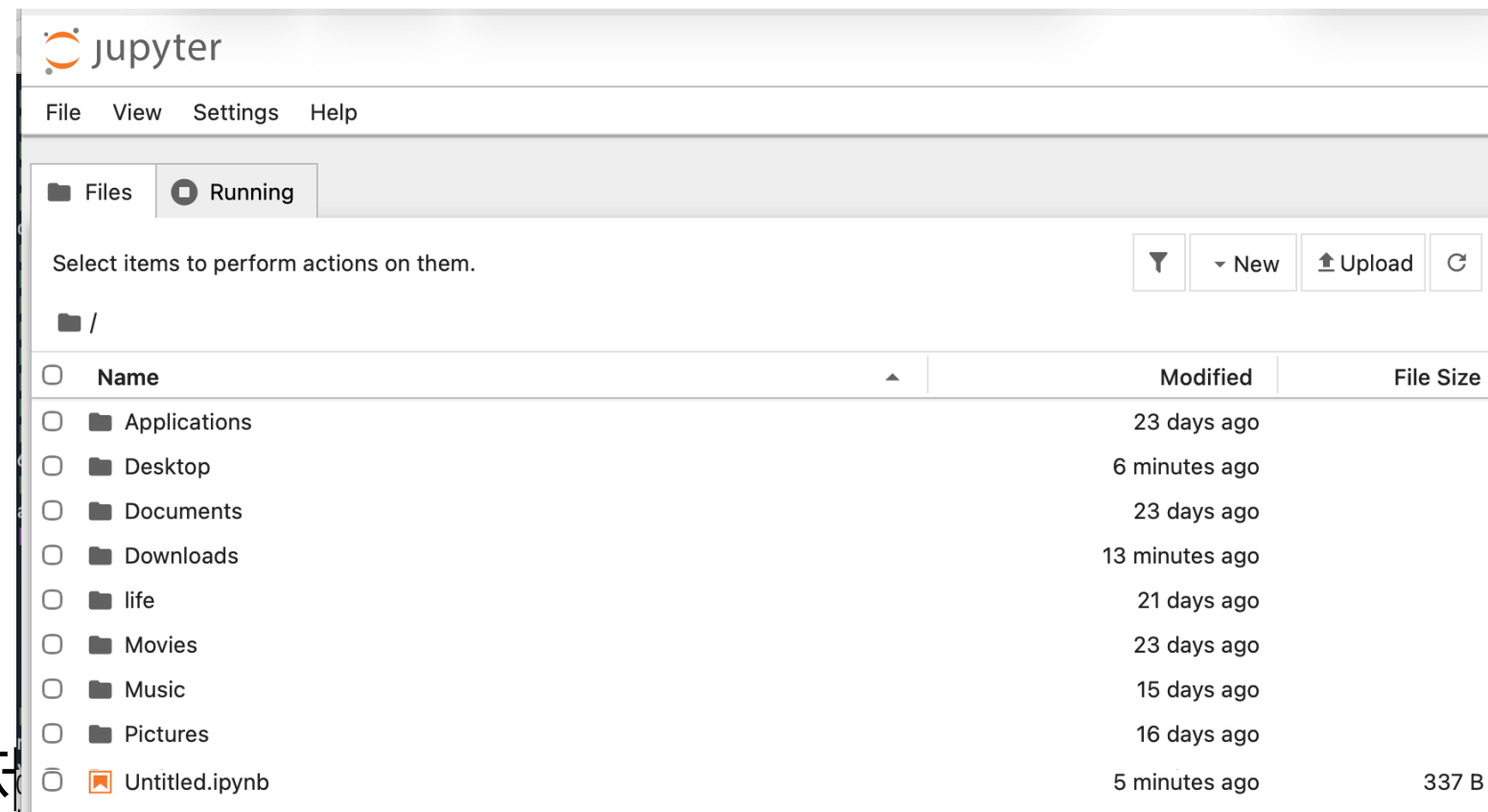
```
pip install torch==1.9.0 torchvision==0.10.0 torchaudio==0.9.0
```

주피터 노트북 설치: `pip install jupyter notebook`

가상 환경에서 `jupyter notebook` 입력해서 실행 가능

구글 코랩:

- 클라우드 기반의 무료 주피터 노트북 개발 환경
- 구글 드라이브 / 도커 / 리눅스 / 구글 클라우드로 구성
- 대량의 데이터 분석 시 필요한 고성능 PC 또는 서버 자원 제공



2.4 파이토치 코드 맛보기



#2.4 딥러닝 학습 알고리즘

1. 데이터셋

gilbutITbook / 080289 Public

<> Code

Issues 10

Pull requests

Actions

Pr

main

080289 / chap02 / data /

gilbutitbook initial release

Name	Last commit message
..	
car_evaluation.csv	initial release

	price	maint	doors	persons	lug_capacity	safety	output
1							
2	vhigh	vhigh	2	2	small	low	unacc
3	vhigh	vhigh	2	2	small	med	unacc
4	vhigh	vhigh	2	2	small	high	unacc
5	vhigh	vhigh	2	2	med	low	unacc

2. 코드 실습

코랩 런타임에 car_evaluation.csv 저장하고 실습 진행
필요한 라이브러리/패키지 호출

[1]
✓ 1초

```
import torch
import torch.nn as sns
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

내려받은 데이터: car_evaluation.csv 불러오기

[2]
✓ 0초

```
dataset = pd.read_csv('/content/car_evaluation.csv')
dataset.head()
```

▼

...	price	maint	doors	persons	lug_capacity	safety	output
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

다음 단계: [dataset 변수로 코드 생성](#) [New interactive sheet](#)

pd.read_csv() 메서드: 해당 경로에 위치한 csv파일 불러오기
dataset.head(): 처음 n개 데이터 확인. / dataset.tail() 마지막 데이터 n개

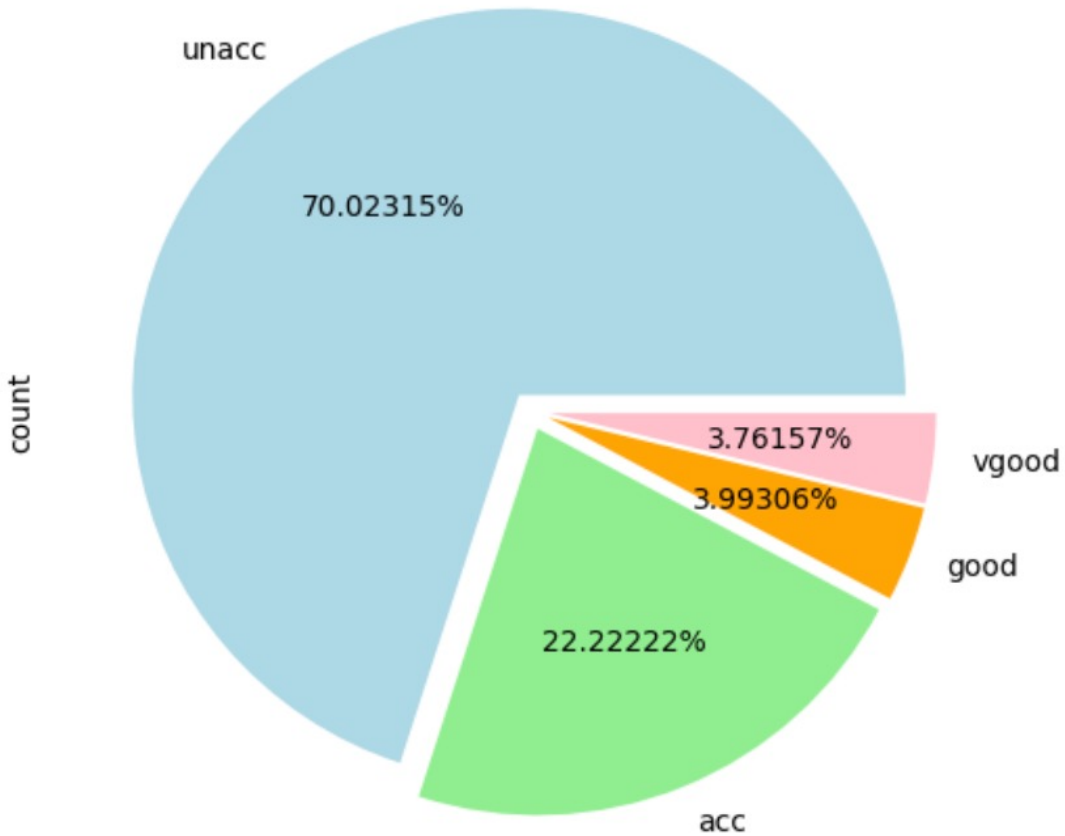
#2.4 딥러닝 학습 알고리즘

2. 코드 실습

주어진 데이터셋 분포 형태로 시각화

```
fig_size = plt.rcParams["figure.figsize"]
fig_size[0] = 8
fig_size[1] = 6
plt.rcParams["figure.figsize"] = fig_size
dataset.output.value_counts().plot(kind='pie', autopct='%0.05f%%',
colors=[ 'lightblue', 'lightgreen', 'orange', 'pink'], explode=(0.05, 0.05, 0.05, 0.05))
```

... <Axes: ylabel='count'>



데이터 전처리: 통계 알고리즘을 기반으로 하는 딥러닝은 단어 -> 숫자(텐서) 변환

필요

예제 데이터셋의 칼럼들은 모두 범주형 타입으로 변환:

범주형 데이터 -> dataset[category] -> 넘파이배열(NumPy array) -> 텐서(Tensor)

astype()메서드로 범주형 변환

```
category = ['price', 'maint', 'doors', 'persons', 'lug_capacity', 'safety'] #예제 데이터셋 칼럼들의 목록
for category in category:
    dataset[category] = dataset[category].astype('category') #astype() 메서드를 이용하여 데이터를 범주형으로 변환

#범주형 데이터를 텐서로 변환: 범주형 데이터 -> dataset[category] -> 넘파이 배열 (NumPy array) -> 텐서 (Tensor)
price = dataset['price'].cat.codes.values
maint = dataset['maint'].cat.codes.values
doors = dataset['doors'].cat.codes.values
persons = dataset['persons'].cat.codes.values
lug_capacity = dataset['lug_capacity'].cat.codes.values
safety = dataset['safety'].cat.codes.values

#2개 이상의 넘파이 객체를 합치기 위해 np.stack사용
categorical_data = np.stack([price, maint, doors, persons, lug_capacity, safety], 1)
categorical_data[:10] #합친 넘파이 배열 중 10개의 행 출력
```

```
... array([[3, 3, 0, 0, 2, 1],
          [3, 3, 0, 0, 2, 2],
          [3, 3, 0, 0, 2, 0],
          [3, 3, 0, 0, 1, 1],
          [3, 3, 0, 0, 1, 2],
          [3, 3, 0, 0, 1, 0],
          [3, 3, 0, 0, 0, 1],
          [3, 3, 0, 0, 0, 2],
          [3, 3, 0, 0, 0, 0],
          [3, 3, 0, 1, 2, 1]], dtype=int8)
```

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

넘파이 배열 -> 텐서

Torch 모듈로 배열을 텐서로 변환

```
▶ categorical_data = torch.tensor(categorical_data, dtype=torch.int64)
categorical_data[:10]

... tensor([[3, 3, 0, 0, 2, 1],
           [3, 3, 0, 0, 2, 2],
           [3, 3, 0, 0, 2, 0],
           [3, 3, 0, 0, 1, 1],
           [3, 3, 0, 0, 1, 2],
           [3, 3, 0, 0, 1, 0],
           [3, 3, 0, 0, 0, 1],
           [3, 3, 0, 0, 0, 2],
           [3, 3, 0, 0, 0, 0],
           [3, 3, 0, 1, 2, 1]])
```

레이블(outputs)로 사용할 칼럼에 대해서도 텐서로 변환
이번에는 get_dummies를 이용하여 넘파이 배열로 변환

```
▶ outputs = pd.get_dummies(dataset.output) #get_dummies: 가변수(dummy variable)로 만들어 주는 함수
outputs = outputs.values
outputs = torch.tensor(outputs).flatten() #1차원 텐서로 변환

print(categorical_data.shape)
print(outputs.shape)

... torch.Size([1728, 6])
torch.Size([6912])
```

텐서로 변환한 범주형 데이터와 레이블에 대한 형태

get_dummies

	gender	weight	nation
0	male	72	Japan
1	female	55	Korea
2	male	68	Australia

pd.get_dummies(df)

weight: 원래 숫자 값 -> 변함X
성별과 국적 데이터는 0,1로 변경

	weight	gender_female	gender_male	nation_Australia	nation_Japan	nation_Korea
0	72	0	1	0	1	0
1	55	1	0	0	0	1
2	68	0	1	1	0	0

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

넘파이 배열 -> 텐서

단일 숫자로 변환된 넘파이 배열을 N차원으로 변경하여 사용 -> 배열을 N차원으로 변환
(모든 범주형 칼럼의 고유 값 수, 차원의 크기) 형태로 배열 생성

```
categorical_column_sizes = [len(dataset[column].cat.categories) for column in categorical_columns]
categorical_embedding_sizes = [(col_size, min(50, (col_size+1)//2)) for col_size in categorical_column_sizes]
print(categorical_embedding_sizes)
```

```
[(4, 2), (4, 2), (4, 2), (3, 2), (3, 2), (3, 2)]
```

칼럼의 고유 값 수를 2로 나누는 것을 많이 사용함.

ex) price 칼럼은 4개의 고유값을 갖음->임베딩 크기: $4 / 2 = 2$

데이터셋을 훈련 / 테스트로 분류

[10]
✓ 0초

```
total_records = 1728
test_records = int(total_records * .2) #전체 데이터 중 20%를 테스트 용도로 사용

categorical_train_data = categorical_data[:total_records - test_records]
categorical_test_data = categorical_data[total_records - test_records:total_records]
train_outputs = outputs[:total_records - test_records]
test_outputs = outputs[total_records - test_records:total_records]
```

[14]
✓ 0초

```
print(len(categorical_train_data))
print(len(train_outputs))
print(len(categorical_test_data))
print(len(test_outputs))
```

▼

```
1383
1383
345
345
```

훈련 및 테스트 용도의 레코드 개수
출력

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

모델 네트워크 생성

```
class Model(nn.Module): #class형태로 구현되는 모델은 nn.Module상속
def __init__(self, embedding_size, output_size, layers, p=0.4): #__init__()은 모델에서 사용될 파라미터와 신경망 초기화 용도. 객체 생성 시 자동호출
    super().__init__() #super().__init__()은 부모 클래스(Model 클래스)에 접근할 때 사용, super는 self 사용하지 않도록 주의
    self.all_embeddings= nn.ModuleList([nn.Embedding(ni, nf) for ni, nf in embedding_size])
    self.embedding_dropout = nn.Dropout(p)

    all_layers= []
    num_categorical_cols = sum((nf for ni, nf in embedding_size))
    input_size = num_categorical_cols #입력층의 크기를 찾기 위해 범주형 칼럼 개수를 input_size변수에 저장
    """모델의 네트워크 계층을 구축하기 위해 for문 이용-> 각 계층을 all_layer목록에 추가
    추가된 계층: Linear(선형계층) - 입력 데이터에 선형 변환( $y = Wx + b$ )
                ReLU: 활성화 함수로 사용
                BatchNorm1d: 배치 정규화 용도로 사용
                Dropout:과적합 방지에 사용 """
    for i in layers:
        #
        all_layers.append(nn.Linear(input_size, i))
        all_layers.append(nn.ReLU(inplace = True))
        all_layers.append(nn.BatchNorm1d(i))
        all_layers.append(nn.Dropout(p))
        input_size = i

    all_layers.append(nn.Linear(layers[-1], output_size))
    self.layers = nn.Sequential(*all_layers) #신경망의 모든 계층이 순차적으로 실행되도록 모든 계층에 대한 목록(all_layers)를 nn.Sequential클래스로 전달

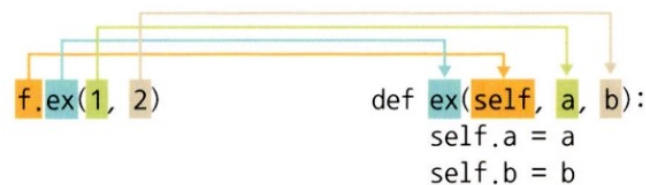
    """ 학습 데이터를 입력받아 forward()함수로 연산 진행. 모델 객체 데이터와 함께 호출하여 자동 실행"""
def forward(self, x_categorical):
    embeddings=[]
    for i, e in enumerate(self.all_embeddings):
        embeddings.append(e(x_categorical[:,i]))

    x = torch.cat(embeddings, 1) #넘파이의 concatenate와 같이만 대상이 텐서임
    x = self.embedding_dropout(x)
    x = self.layers(x)
    return x
```

```
def __init__(self, embedding_size, output_size, layers, p=0.4)
                (a)         (b)         (c)         (d)         (e)
```

① self: 첫 번째 파라미터는 self를 지정해야 하며 자기 자신을 의미합니다. 예를 들어 ex라는 함수가 있을 때 self 의미는 다음 그림과 같습니다.

▼ 그림 2-31 self 의미



② embedding_size: 범주형 칼럼의 임베딩 크기

③ output_size: 출력층의 크기

④ layers: 모든 계층에 대한 목록

⑤ p: 드롭아웃(기본값은 0.5)

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

모델 네트워크 생성

모델 훈련을 위해 앞에서 정의했던 Model클래스의 객체 생성 ->

(범주형 칼럼의 임베딩 크기 , 출력 크기 , 은닉층의 뉴런 , 드롭아웃

) 전달

```
model = Model(categorical_embedding_sizes, 4, [200, 100, 50], p=0.4)
print(model)

... Model(
  (all_embeddings): ModuleList(
    (0-2): 3 x Embedding(4, 2)
    (3-5): 3 x Embedding(3, 2)
  )
  (embedding_dropout): Dropout(p=0.4, inplace=False)
  (layers): Sequential(
    (0): Linear(in_features=12, out_features=200, bias=True)
    (1): ReLU(inplace=True)
    (2): BatchNorm1d(200, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Dropout(p=0.4, inplace=False)
    (4): Linear(in_features=200, out_features=100, bias=True)
    (5): ReLU(inplace=True)
    (6): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): Dropout(p=0.4, inplace=False)
    (8): Linear(in_features=100, out_features=50, bias=True)
    (9): ReLU(inplace=True)
    (10): BatchNorm1d(50, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Linear(in_features=50, out_features=4, bias=True)
  )
)
```

← 모델 구조

모델 훈련시키기 전 손실 함수, 옵티마이저 정의

[24]
✓ 7초

```
loss_function = nn.CrossEntropyLoss() #데이터 분류 크로스 엔트로피 손실 함수 사용
optimizer = torch.optim.Adam(model.parameters(), lr=0.001) #Adam옵티마이저 사용
```

파이토치는 GPU에 최적화된 딥러닝 프레임워크 but GPU없으면 CPU사용할 수 있도록 지정

[26]
✓ 0초

```
if torch.cuda.is_available():
    device = torch.device("cuda")
else:
    device = torch.device("cpu")
```

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

모델 학습

```
▶ epochs = 500
aggregated_losses = []
train_outputs = train_outputs.to(device = device, dtype = torch.int64)

for i in range(epochs):    #for문 500회 반복, 반복마다 손실함수가 오차 계산
    i+=1
    y_pred = model(categorical_train_data).to(device)
    single_loss = loss_function(y_pred, train_outputs)
    aggregated_losses.append(single_loss)    #반복할 때마다 오차를 aggregated_losses에 추가

    if i%25 ==1:
        print(f'epoch: {i:3} loss: {single_loss.item():10.8f}')

    optimizer.zero_grad()
    single_loss.backward()    #가중치를 업데이트하기 위해 손실 함수의 backward()메서드 호출
    optimizer.step()    #옵티마이저 함수의 step()메서드를 이용하여 기울기 업데이트

print(f'epoch: {i:3} loss: {single_loss.item():10.10f}')    #오차가 25에포크마다 출력
```

```
... epoch:   1 loss: 1.61936343
epoch:  26 loss: 1.42771447
epoch:  51 loss: 1.34080112
epoch:  76 loss: 1.21769702
epoch: 101 loss: 1.11889493
epoch: 126 loss: 0.96221215
epoch: 151 loss: 0.83626074
epoch: 176 loss: 0.74043828
epoch: 201 loss: 0.70440668
epoch: 226 loss: 0.65792739
epoch: 251 loss: 0.63422424
epoch: 276 loss: 0.62840635
epoch: 301 loss: 0.61672682
epoch: 326 loss: 0.59299761
epoch: 351 loss: 0.58877850
epoch: 376 loss: 0.58461779
epoch: 401 loss: 0.58425850
epoch: 426 loss: 0.57881188
epoch: 451 loss: 0.57344252
epoch: 476 loss: 0.56962287
epoch: 500 loss: 0.5687233806
```

#2.4 딥러닝 학습 알고리즘

2. 코드 실습

예측

```
[28]
✓ 0초
test_outputs = test_outputs.to(device=device, dtype = torch.int64)
with torch.no_grad():
    y_val = model(categorical_test_data)
    loss = loss_function(y_val, test_outputs)
    print(f'Loss: {loss: 8f}')
```

✓ Loss: 0.555954

Loss값은 훈련 데이터셋의 손실값과 유사 -> 과적합X

모델의 예측 확인

```
[29]
✓ 0초
print(y_val[:5])

tensor([[ 3.1941,  2.1203, -3.0177, -2.8242],
        [ 2.3504,  1.3578, -2.7744, -2.5725],
        [ 3.3418,  2.1205, -3.6796, -3.6553],
        [ 2.9272,  1.5873, -3.7122, -3.5792],
        [ 4.1478,  2.5923, -4.7767, -4.5337]])
```

모델의 처음 5개 값 출력:
4개 클래스 각각에 대한 점수를 출력
가장 큰 값의 위치(인덱스)= 모델의 예측 클래스

가장 큰 값을 갖는 인덱스 확인

```
[30]
✓ 0초
y_val = np.argmax(y_val, axis=1)
print(y_val[:5])

tensor([0, 0, 0, 0, 0])
```

모델의 처음 5개 값 모두 0번째 인덱스 > 나머지 인덱스 점수
ex) 3.1941 > 2.1203, -3.0177, -2.8242

출력 결과 모두 인덱스 0이 출력 -> 인덱스가 0인 값 > 인덱스가 1인 값 : 처리된 출력이 0

#2.4 딥러닝 학습 알고리즘

3. 성능 평가 지표

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

정확도 (accuracy) : 전체 (초)

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
print(confusion_matrix(test_outputs, y_val))
print(classification_report(test_outputs, y_val))
print(accuracy_score(test_outputs, y_val))
```

재현율 (recall) : 실제 posi

```
... [[259  0]
     [ 86  0]]
```

정밀도 (precision) : 모델(

F1-스코어 : 서로 trade-off

```
precision recall f1-score support
0 0.75 1.00 0.86 259
1 0.00 0.00 0.00 86

accuracy 0.75 345
macro avg 0.38 0.50 0.43 345
weighted avg 0.56 0.75 0.64 345

0.7507246376811594
```